الاحتراف الكامل

في مكتبات بايثون لعلم البيانات والتعلّم الآلي

الجزء الثامن: مكتبة Plotly

الفصول ٧ - ١١: الأمثلة العملية، الأداء، التصحيح، أفضل الممارسات، والمشاريع المصغّرة

مرجع احترافي للمبرمجين والمهندسين وعلماء البيانات

جدول المحتويات

جدول المحتويات 2
الفصل السابع: الأمثلة العملية 4
٧.١ المستوى المبتدئ 4
مثال ١: رسم تشتت تفاعلي بسيط 4
٧.٢ المستوى المتوسط 4
مثال ٢: رسم سلسلة زمنية مع Range Slider 4
٧.٣ المستوى الاحترافي 4
مثال ٣: لوحة معلومات مركّبة عبر make_subplots بأنواع رسوم مختلطة 4
٧.٤ مثال من مشروع حقيقي: لوحة Dash كاملة بفلتر تفاعلي 5
تمارين عملية 5
حلول مقترحة (تلميحات) 6
الفصل الثامن: الأداء 7
٨.١ حجم ملف HTML الناتج 7
٨.٢ استراتيجيات تقليل الحجم 7
أفضل الممارسات 7
الأخطاء الشائعة 7
الفصل التاسع: التصحيح 8
٩.١ الرسم لا يظهر إطلاقًا 8
٩.٢ فحص بنية Figure مباشرة 8
٩.٣ رسائل خطأ شائعة 8
الأخطاء الشائعة 8
الأسئلة الشائعة 8
الفصل العاشر: أفضل الممارسات 9
١٠.١ أسلوب الكتابة الاحترافي 9
١٠.٢ هيكلة تطبيقات Dash 9
١٠.٣ الاختبار 9
أفضل الممارسات 9
الفصل الحادي عشر: مشاريع مصغّرة 10
مشروع ١: مستكشف بيانات تفاعلي بفلاتر متعددة (Dash) 10
مشروع ٢: لوحة سلسلة زمنية مالية مع مؤشرات 10
مشروع ٣: خريطة جغرافية تفاعلية بسيطة 11
تمارين عملية 11
حلول مقترحة (تلميحات) 11

مراجع وروابط رسمية 11
خاتمة Plotly 11

الفصل السابع: الأمثلة العملية

٧.١ المستوى المبتدئ

مثال ١: رسم تشتت تفاعلي بسيط

```
import plotly.express as px
import pandas as pd

df = pd.DataFrame({
    'age': [25, 30, 35, 40, 45],
    'salary': [4000, 5500, 6200, 7000, 7500],
    'department': ['IT', 'IT', 'Sales', 'Sales', 'HR'],
})

fig = px.scatter(df, x='age', y='salary', color='department', title='الراتب حسب العمر والقسم')
fig.show()
```

شرح الكود: تمرير عمود department إلى color يجعل Plotly تلوّن النقاط تلقائيًا حسب الفئة وتضيف legend تفاعلي يمكن النقر عليه لإخفاء/إظهار كل قسم على حدة.

٧.٢ المستوى المتوسط

مثال ٢: رسم سلسلة زمنية مع Range Slider

```
df = pd.DataFrame({
    'date': pd.date_range('2023-01-01', periods=365),
    'price': (pd.Series(range(365)).apply(lambda i: 100 + i * 0.5 + (i % 30))),
})

fig = px.line(df, x='date', y='price', title='سعر السهم عبر عام كامل')
fig.update_xaxes(rangeslider_visible=True)
fig.show()
```

شرح الكود: rangeslider_visible=True يضيف شريط تمرير أسفل الرسم يتيح للمستخدم اختيار نطاق زمني معيّن بالسحب، وهي ميزة مهمة جدًا للسلاسل الزمنية الطويلة الشائعة في التطبيقات المالية.

٧.٣ المستوى الاحترافي

مثال ٣: لوحة معلومات مركّبة عبر make_subplots بأنواع رسوم مختلطة

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(
    rows=1, cols=2,
    specs=[[{'type': 'xy'}, {'type': 'domain'}]],   # عمود يسار خطي، عمود يمين دائري
    subplot_titles=('توزيع الفئات', 'الإيراد الشهري'),
)
```

```
fig.add_trace(go.Scatter(x=df['date'], y=df['price'], mode='lines'), row=1, col=1)
fig.add_trace(go.Pie(labels=['A','B','C'], values=[30,45,25]), row=1, col=2)

fig.update_layout(showlegend=False, title_text='لوحة تحليل شاملة')
fig.show()
```

شرح الكود: specs في make_subplots ضروري عند دمج أنواع رسوم مختلفة الطبيعة (مثل رسم خطي يعتمد على محاور xy، مع رسم دائري لا يعتمد على محاور إطلاقًا بل على نوع domain)، وهو تفصيل تقني يغفل عنه كثير من المبتدئين فيحصلون على خطأ عند الجمع بين نوعين مختلفين دون تحديده.

٧.٤ مثال من مشروع حقيقي: لوحة Dash كاملة بفلتر تفاعلي

```
from dash import Dash, dcc, html, Input, Output
import plotly.express as px
import pandas as pd

df = pd.DataFrame({
    'date': pd.date_range('2024-01-01', periods=200),
    'region': (['North', 'South'] * 100),
    'sales': range(200),
})

app = Dash(__name__)
app.layout = html.Div([
    dcc.Dropdown(df['region'].unique(), 'North', id='region-filter'),
    dcc.Graph(id='sales-graph'),
])

@app.callback(Output('sales-graph', 'figure'), Input('region-filter', 'value'))
def update_graph(selected_region):
    filtered = df[df['region'] == selected_region]
    return px.line(filtered, x='date', y='sales', title=f'مبيعات منطقة
{selected_region}')

if __name__ == '__main__':
    app.run(debug=True)
```

شرح الكود: هذا مثال كامل على النمط الأساسي لكل تطبيق Dash: عنصر إدخال (Dropdown)، عنصر عرض (Graph)، ودالة callback تربط بينهما تلقائيًا — كلما تغيّر الفلتر يُعاد بناء Figure المناسب ديناميكيًا في الخادم ويُرسَل للمتصفح.

تمارين عملية

١. ارسم رسمًا بخريطة حرارية (px.imshow) لمصفوفة ارتباط (Correlation Matrix) بين عدة أعمدة رقمية.

٢. أضف Range Slider وأزرار اختيار سريعة للفترة (١ شهر، ٦ أشهر، سنة) لرسم سلسلة زمنية عبر update_xaxes(rangeselector=...).

٣. ابنِ تطبيق Dash بسيطًا بفلتر Dropdown لعرض توزيع مبيعات فئة مختارة.

حلول مقترحة (تلميحات)

١. corr = df.corr(numeric_only=True); px.imshow(corr, text_auto=True)

٢. استخدم .fig.update_xaxes(rangeslider_visible=True, rangeselector=dict(buttons=[...]))

٣. اتبع نمط مثال ٤: Dropdown + Graph + callback واحد يُصفّي DataFrame ويُعيد px.histogram.

الفصل الثامن: الأداء

٨.١ حجم ملف HTML الناتج

كل رسم Plotly تفاعلي يحمل معه بيانات كل نقطة داخل ملف HTML الناتج (أو خلية Jupyter)، مما يعني أن رسم مئات الآلاف من النقاط قد ينتج ملفات ضخمة بطيئة الفتح في المتصفح، على عكس Matplotlib التي تنتج صورة نقطية ثابتة بحجم أصغر بكثير بغض النظر عن عدد النقاط.

٨.٢ استراتيجيات تقليل الحجم

الطريقة	الاستراتيجية
df.sample(n=5000) أو أخذ كل نقطة عاشرة قبل الرسم	تخفيض العيّنة (Downsampling)
go.Scattergl يستخدم تسريع WebGL، أسرع بكثير لآلاف النقاط	استخدام Scattergl بدل Scatter
رسم متوسطات يومية بدل كل نقطة خام في بيانات عالية التردد	تجميع البيانات قبل الرسم
تحميل مكتبة plotly.js من الإنترنت بدل تضمينها كاملة في كل ملف HTML	'include_plotlyjs='cdn عند التصدير

```
import plotly.graph_objects as go

fig = go.Figure(go.Scattergl(x=x_large, y=y_large, mode='markers'))  # WebGL لبيانات ضخمة

fig.write_html('report.html', include_plotlyjs='cdn')   # ملف أصغر بكثير
```

أفضل الممارسات

- استخدم go.Scattergl بدل go.Scatter عند رسم أكثر من بضعة آلاف نقطة.
- خفّض العيّنة أو جمّع البيانات (Aggregation) قبل الرسم للسلاسل الزمنية عالية التردد.
- استخدم 'include_plotlyjs='cdn عند تصدير عدة ملفات HTML لتفادي تكرار تضمين المكتبة كاملة في كل ملف.

الأخطاء الشائعة

- رسم ملايين النقاط عبر px.scatter العادي (يستخدم SVG افتراضيًا) بدل Scattergl المسرّع بـ WebGL.
- تضمين plotly.js كاملة (include_plotlyjs=True الافتراضي) في عشرات الملفات المُصدَّرة بدل استخدام cdn مرة واحدة.

الفصل التاسع: التصحيح

٩.١ الرسم لا يظهر إطلاقًا

السبب الشائع	المشكلة
renderer غير مضبوط بشكل صحيح لبيئة Jupyter الحالية (كلاسيكي مقابل JupyterLab مقابل VS Code)	لا شيء يظهر في Jupyter
نسيان تحديد specs الصحيح لكل خلية في make_subplots	خطأ عند دمج أنواع رسوم مختلفة في subplots
عدم تثبيت مكتبة kaleido المطلوبة للتصدير كصورة ثابتة	write_image يفشل برسالة عن kaleido
عدد نقاط ضخم مع Scatter العادي بدل Scattergl	الرسم بطيء جدًا أو المتصفح يتجمّد

٩.٢ فحص بنية Figure مباشرة

```
print(fig.data)              # كل الـ Traces الموجودة فعليًا
print(fig.layout)            # كامل إعدادات التخطيط الحالية
fig.show(renderer='json')    # طباعة تمثيل JSON كامل للتشخيص العميق
```

٩.٣ رسائل خطأ شائعة

رسالة الخطأ	السبب
ValueError: Invalid property specified for object of ...type	تمرير معامل غير موجود لنوع Trace معيّن (كل نوع له معاملاته الخاصة)
ImportError: Please install kaleido	استدعاء write_image أو to_image دون تثبيت kaleido
'subplot type 'xy' is not compatible with 'domain trace type	محاولة رسم Pie/Sunburst في خلية subplot لم تُحدَّد لـ type: 'domain'

الأخطاء الشائعة

- الخلط بين معاملات express (أسماء أعمدة كنصوص) ومعاملات graph_objects (قيم مباشرة)، فهما نمطان مختلفان تمامًا رغم التشابه الظاهري في النتيجة.
- تجاهل رسالة خطأ kaleido عند write_image بدل تثبيت المكتبة المطلوبة مباشرة.

الأسئلة الشائعة

س: لماذا يظهر رسمي فارغًا في VS Code رغم عدم وجود خطأ؟

ج: غالبًا بسبب renderer غير مناسب؛ جرّب pio.renderers.default = 'vscode' أو تأكد من تثبيت إضافة Jupyter المناسبة.

الفصل العاشر: أفضل الممارسات

١٠.١ أسلوب الكتابة الاحترافي

- ابدأ دائمًا بـ Plotly Express لأي رسم قياسي، ولا تنتقل لـ graph_objects إلا عند الحاجة الفعلية.
- غلّف منطق بناء الرسوم المتكررة في دوال مخصصة تُرجع Figure جاهزًا.
- استخدم Templates موحدة عبر كل رسوم المشروع لتناسق بصري احترافي.
- افصل منطق تجهيز البيانات (Pandas) عن منطق بناء الرسم (Plotly) في دوال أو وحدات منفصلة.

```
def make_sales_chart(df: pd.DataFrame, region: str):
    filtered = df[df['region'] == region]
    fig = px.line(filtered, x='date', y='sales', title=f'مبيعات {region}')
    fig.update_layout(template='plotly_white')
    return fig
```

١٠.٢ هيكلة تطبيقات Dash

```
app/
├── app.py              # نقطة الدخول وتعريف layout الرئيسي
├── callbacks.py        # كل دوال @app.callback
├── charts.py           # دوال بناء الرسوم (px/go)
└── data.py             # تحميل وتجهيز البيانات
```

فصل دوال بناء الرسوم (charts.py) عن دوال callbacks.py يجعل كل رسم قابلاً للاختبار المستقل ولإعادة الاستخدام في أكثر من صفحة داخل التطبيق نفسه.

١٠.٣ الاختبار

```
def test_make_sales_chart_has_one_trace():
    fig = make_sales_chart(sample_df, region='North')
    assert len(fig.data) == 1
    assert fig.data[0].type == 'scatter'
```

أفضل الممارسات

- اكتب دوال بناء رسوم تقبل DataFrame ومعاملات وتُرجع Figure جاهزًا بدل كود رسم متكرر مبعثر.
- استخدم Templates موحدة عبر المشروع.
- افصل بين data.py وcharts.py وcallbacks.py في أي تطبيق Dash متوسط الحجم أو أكبر.

الفصل الحادي عشر: مشاريع مصغّرة

مشروع ١: مستكشف بيانات تفاعلي بفلاتر متعددة (Dash)

```
from dash import Dash, dcc, html, Input, Output
import plotly.express as px
import pandas as pd, numpy as np

rng = np.random.default_rng(5)
df = pd.DataFrame({
    'category': rng.choice(['A','B','C'], 300),
    'value': rng.normal(50, 15, 300),
    'date': pd.date_range('2024-01-01', periods=300),
})

app = Dash(__name__)
app.layout = html.Div([
    dcc.Checklist(df['category'].unique(), df['category'].unique().tolist(), id='cat-
filter'),
    dcc.Graph(id='main-chart'),
])

@app.callback(Output('main-chart', 'figure'), Input('cat-filter', 'value'))
def update(selected_categories):
    filtered = df[df['category'].isin(selected_categories)]
    return px.box(filtered, x='category', y='value', color='category')

if __name__ == '__main__':
    app.run(debug=True)
```

يطبّق هذا المشروع: dcc.Checklist للفلترة متعددة الاختيار، وisin للتصفية المرتبطة بها، وpx.box لعرض توزيع القيم إحصائيًا لكل فئة مختارة بشكل تفاعلي فوري عند تغيير الفلتر.

مشروع ٢: لوحة سلسلة زمنية مالية مع مؤشرات

```
dates = pd.date_range('2024-01-01', periods=250)
price = 100 + np.cumsum(rng.normal(0, 2, 250))
df = pd.DataFrame({'date': dates, 'price': price})
df['ma20'] = df['price'].rolling(20).mean()   # متوسط متحرك 20 يومًا

fig = go.Figure()
fig.add_trace(go.Scatter(x=df['date'], y=df['price'], name='السعر',
line=dict(color='#2E5395')))
fig.add_trace(go.Scatter(x=df['date'], y=df['ma20'], name='20 المتوسط المتحرك',
line=dict(dash='dash')))
fig.update_layout(title='تحليل سعر السهم', xaxis_rangeslider_visible=True)
```

يطبّق هذا المشروع: rolling من Pandas لحساب مؤشر فني شائع، ثم تمثيله كـ Trace إضافي فوق السعر الخام في نفس الرسم، وهو نمط أساسي جدًا في التحليل المالي التفاعلي.

مشروع ٣: خريطة جغرافية تفاعلية بسيطة

```
geo_df = pd.DataFrame({
    'country': ['Egypt', 'Saudi Arabia', 'UAE', 'Morocco'],
    'iso_alpha': ['EGY', 'SAU', 'ARE', 'MAR'],
    'value': [120, 95, 80, 60],
})

fig = px.choropleth(
    geo_df, locations='iso_alpha', color='value',
    hover_name='country', color_continuous_scale='Blues',
    title='مؤشر تجريبي حسب الدولة',
)
fig.show()
```

يطبّق هذا المشروع: px.choropleth لرسم خريطة تلوين جغرافي (Choropleth Map) بسطر واحد، مستخدمًا رموز الدول القياسية ISO Alpha-3، وهي أداة قوية جدًا لعرض بيانات جغرافية موزّعة بشكل احترافي وتفاعلي.

تمارين عملية

١. أضف لمشروع ١ عنصر dcc.RangeSlider لتصفية القيم ضمن نطاق رقمي إضافي.

٢. أضف مؤشرًا فنيًا ثانيًا (متوسط متحرك ٥٠ يومًا) لمشروع ٢ وقارنه بصريًا مع مؤشر الـ ٢٠ يومًا.

٣. وسّع خريطة مشروع ٣ لتشمل ١٠ دول إضافية وأضف animation_frame لعرض تغيّر القيمة عبر سنوات وهمية متعددة.

حلول مقترحة (تلميحات)

١. أضف dcc.RangeSlider(min=df['value'].min(), max=df['value'].max()) وربطه بـ callback إضافي أو ادمجه في نفس الدالة.

٢. df['ma50'] = df['price'].rolling(50).mean() ثم أضف Trace ثالث بنفس الطريقة.

٣. أضف عمود year للبيانات الجغرافية الوهمية ومرر 'animation_frame='year لـ px.choropleth.

مراجع وروابط رسمية

- التوثيق الرسمي: https://plotly.com/python/
- توثيق Dash: https://dash.plotly.com/
- الكود المصدري: https://github.com/plotly/plotly.py

خاتمة Plotly

بهذا نُتم مرجعًا احترافيًا كاملاً لـ Plotly، مغطّين الرسم التفاعلي من الأساسيات حتى بناء تطبيقات Dash كاملة. المكتبة التالية هي Scikit-learn، حيث سننتقل من عرض البيانات إلى بناء واجهات نمذجة قياسية موحّدة، مع التركيز الكامل على تصميم واجهاتها البرمجية دون الخوض في الخوارزميات نفسها.